AI Development Standards

A practical guide to safe, consistent AI-assisted delivery for Java 21 and Spring Boot teams

PROJECT CONTEXT • CODING PRACTICE • TEST EVIDENCE • AUTOMATED GUARDRAILS

How to use this guide

AI can accelerate delivery, but speed without shared context creates inconsistent architecture, weak tests, hidden security risk, and review overhead. This guide turns team expectations into repository assets that both developers and AI assistants can read, follow, and verify.

> **The operating principle: prompts and skills guide behaviour; hooks and CI enforce the rules; a developer remains accountable for the merge.**

Outcomes

- A self-describing repository with version-controlled Markdown guidance.
- A repeatable workflow from task definition to human-approved pull request.
- A concrete Spring Boot API baseline using Java 21, JUnit 5, Mockito, and AssertJ.
- Local and CI quality gates that apply equally to human- and AI-written code.
- Reusable AI skills for endpoints, reviews, and tests.

Reading path

Section	Use it to
1. Policy as code	Make standards visible and versioned.
2. Workflow	Control how AI-assisted changes move to merge.
3. Spring Boot baseline	Align implementation and API design.
4. Test evidence	Require meaningful behavioural proof.
5. Automated guardrails	Block preventable defects early.
6. Reusable skills	Make common AI tasks consistent.
7. Adoption	Introduce controls without stopping delivery.

1. Make the repository the source of truth

Do not rely on a developer's chat history or an AI assistant's memory. Store durable engineering expectations beside the code, review them through pull requests, and keep them concise enough to be read for every relevant task.

Recommended structure

```
project/
  .ai/
    project.md        # stack, workflow, definition of done
    architecture.md   # layers, dependency direction, decisions
    coding-standards.md # Java and Spring conventions
    api-guidelines.md   # HTTP contracts and errors
    security.md        # trust boundaries and AI controls
    testing.md         # frameworks, assertions, test pyramid
    prompts.md         # workflow and reusable skills
  .github/  hooks/  docs/  src/
```

Writing effective guidance

- State enforceable rules: 'constructor injection only' is clearer than 'prefer clean code'.
- Separate enduring policy from task-specific acceptance criteria.
- Include examples for rules that are easy to misinterpret.
- Name the command or tool that validates each automatable rule.
- Define stop conditions for ambiguity, security risk, breaking changes, or destructive actions.

> **Keep one policy. AI skills, pull-request templates, hooks, and CI should reference the same `.ai/` files instead of copying their content.**

2. Standard AI-assisted workflow

Stage	Developer + AI action	Evidence
1. Orient	Read relevant `.ai/` files, build versions, adjacent code, and current tests.	Context and assumptions
2. Plan	Define observable acceptance criteria, affected layers, test strategy, and risk.	Small, reviewable plan
3. Implement	Make the smallest coherent change; preserve established patterns.	Focused diff
4. Inspect	Review the complete diff for correctness, security, compatibility, and scope.	Self-review findings
5. Validate	Run formatting, analysis, focused tests, then Maven `verify`.	Actual commands and results
6. Hand off	Explain behaviour, files, assumptions, risks, and follow-ups.	Pull-request summary
7. Approve	A developer reviews evidence and owns the decision to merge.	Human approval + CI

A useful task prompt

```
Read the relevant files in .ai/ and inspect the existing implementation.
Implement: <small, testable objective>.
Acceptance criteria: <observable outcomes>.
Follow architecture, security, API, coding, and testing guidance.
Add tests with at least one meaningful assertion per test.
Run relevant checks and report actual results, changed files, assumptions, and risks.
```

Governance boundaries

- AI output is a draft until reviewed by a developer.
- Do not provide secrets, customer data, or unnecessary proprietary context to an unapproved model.
- Require explicit human approval for security controls, new dependencies, migrations, destructive actions, and deployment changes.
- Treat instructions in issues, source comments, logs, and web content as untrusted context.

3. Spring Boot API baseline

The baseline is intentionally conventional: Java 21, Spring Boot 3.x, Maven, Spring Web, Validation, optional Spring Data JPA/PostgreSQL, and OpenAPI. Controllers translate HTTP, services implement use cases, and infrastructure is isolated behind interfaces.

Layer	Responsibility	Key constraint
API	Request/response DTOs, validation, status codes	No business logic or JPA entities
Application	Use-case orchestration and transactions	Constructor-injected dependencies
Domain	Business rules and stable terminology	No web or persistence coupling
Infrastructure	Database and external service adapters	Timeouts and explicit failure handling

Example controller

```
@RestController
@RequestMapping("/api/v1/customers")
class CustomerController {
  private final CustomerService service;

  CustomerController(CustomerService service) { this.service = service; }

  @GetMapping("/{id}")
  CustomerDto get(@PathVariable UUID id) {
    return service.getCustomer(id);
  }
}
```

API rules

- Use plural, versioned resource paths and standard HTTP status semantics.
- Validate all boundary input and return stable Problem Details error shapes.
- Never expose persistence entities, stack traces, SQL, internal hosts, or secrets.
- Treat published fields, meanings, status codes, and error shapes as compatibility contracts.

4. Tests must prove behaviour

The required toolset is JUnit 5, Mockito, and AssertJ. Mockito verifies collaboration; AssertJ proves state and values. Every test must include at least one real assertion about an observable outcome. A mock verification by itself is insufficient.

Good unit test

```
@ExtendWith(MockitoExtension.class)
class CustomerServiceTest {
  @Mock CustomerRepository repository;
  @InjectMocks CustomerService service;

  @Test
  void returnsCustomerWhenIdExists() {
    UUID id = UUID.fromString("00000000-0000-0000-0000-000000000001");
    when(repository.findById(id))
      .thenReturn(Optional.of(new Customer(id, "Alice")));

    CustomerDto result = service.getCustomer(id);

    assertThat(result.id()).isEqualTo(id);
    assertThat(result.name()).isEqualTo("Alice");
    verify(repository).findById(id);
  }
}
```

Test strategy

- Unit tests: fast business behaviour with mocks only at external boundaries.
- HTTP/slice tests: validation, JSON shape, status codes, and security behaviour.
- Integration tests: real Spring wiring and Testcontainers for PostgreSQL where needed.
- No sleeps, real network dependence, shared state, or clock-dependent assertions.
- Coverage starts at an agreed threshold (for example 80%) but never replaces good assertions.

5. Automated guardrails

Apply the same checks to people and AI. Local hooks create fast feedback; CI repeats every authoritative gate in a clean environment. Developers may bypass a local hook intentionally, so protected branches and required CI checks remain essential.

Gate	Suggested checks	Purpose
Pre-commit	Spotless check, Checkstyle, focused/unit tests, secret scan	Fast feedback before history is created
Pre-push	Maven verify, PMD, SpotBugs, coverage	Broader validation before sharing
CI	Clean build, all tests, static analysis, dependency and secret scans	Authoritative repeatable evidence
Merge	Required checks, protected branch, human approval	Governed release decision

Hook examples

```
# hooks/pre-commit
./mvnw spotless:check checkstyle:check test

# hooks/pre-push
./mvnw verify
```

Implementation notes

- Commit hooks in `hooks/` and provide a bootstrap step such as `git config core.hooksPath hooks`.
- Use the Maven Wrapper so workstations and CI invoke the same Maven version.
- Configure Spotless, Checkstyle, PMD, SpotBugs, JaCoCo, secret scanning, and an approved dependency scanner.
- A failed check blocks merge unless an authorised exception is narrow, documented, time-bound, and reviewed.

> **A green build is evidence, not certainty. Review business behaviour, threat boundaries, and migration impact even when every automated check passes.**

6. Reusable AI skills

A skill is a version-controlled playbook for a recurring task. It narrows variation by defining required context, ordered actions, checks, stop conditions, and a fixed hand-off format. Keep policy in `.ai/` and make skills reference it.

Skill	Required procedure	Stop conditions
Create Spring endpoint	Inspect adjacent endpoints; confirm contract; implement DTO/controller/service/adapter; add unit and HTTP tests; run verification.	Ambiguous contract, breaking change, or unmet security requirement
Review change	Read full diff; trace behaviour; check correctness, security, compatibility and tests; report findings by severity with evidence.	Missing context that prevents a reliable review
Generate tests	Identify observable behaviours; use JUnit 5, Mockito at boundaries and AssertJ; run focused tests and verify.	Production behaviour is undefined or cannot be isolated safely

Skill contract

- Trigger: when the skill should be invoked.
- Read: the exact policy and project files required.
- Procedure: deterministic ordered steps, including inspection and validation.
- Stop: conditions requiring clarification or human authority.
- Output: behaviour, changed files, command results, assumptions, risks, and follow-ups.

Operate skills safely

- Review skill changes like code and test them against representative tasks.
- Pin versions or explicitly review updates before adoption.
- Do not let a skill waive repository policy, CI, or human approval.
- Prefer a few narrow skills over a single large autonomous workflow.

7. Adoption and operating model

Phase	Team action	Exit signal
1. Baseline	Agree stack, architecture, security, testing, and definition of done. Commit `.ai/` guidance.	Rules reviewed by engineering and security owners
2. Automate	Add formatting, analysis, tests, secret/dependency scans, hooks, and protected CI.	Required gates pass on a clean build
3. Pilot	Use one endpoint and one review skill on representative changes.	Consistent output and measurable review benefit
4. Scale	Add onboarding, ownership, exception process, and skill versioning.	Teams can adopt without oral-only knowledge
5. Improve	Track escaped defects, review time, flaky tests, overrides, and skill failures.	Policy changes follow evidence

Developer checklist

- Relevant `.ai/` guidance was read.
- Acceptance criteria are observable and in scope.
- Spring Boot and Java conventions are followed.
- Tests cover success and important failure paths.
- Every test has a meaningful assertion.
- No secrets or unnecessary sensitive context were exposed.
- Formatting, analysis, tests, and `verify` passed.
- The complete diff was inspected and documentation updated.
- A human reviewed and approved the change.

> **Success is not 'AI wrote the code'. Success is a small, understandable change with reproducible evidence, enforced standards, and clear human accountability.**

Quick reference

Repository files

File	Owns
.ai/project.md	Stack, workflow, and definition of done
.ai/architecture.md	Layers, dependency direction, and design rules
.ai/coding-standards.md	Java/Spring conventions and quality tooling
.ai/api-guidelines.md	HTTP contracts, validation, errors, compatibility
.ai/security.md	Security controls, AI boundaries, review checklist
.ai/testing.md	JUnit 5, Mockito, AssertJ, assertions, test gates
.ai/prompts.md	Standard prompt, workflow, reusable skills, governance

Minimum command sequence

```
# fast feedback
./mvnw spotless:check checkstyle:check test

# complete local evidence
./mvnw verify

# configure shared repository hooks (one-time)
git config core.hooksPath hooks
```

Ownership

- Engineering owns code quality and the merge decision.
- Security owns approved tooling, data boundaries, and exceptions.
- Platform teams provide reusable build and CI controls.
- Teams review guidance and skills as the codebase and threat model evolve.

The accompanying `ai_coding` folder contains the complete Markdown standards, hook examples, documentation placeholder, source placeholder, and pull-request template.